Smart AI Glasses

By

Ahtsham Ali
22-ARID-3091

Meesam Bilal
22-ARID-3097

Muhammad Khalil Ullah
22-ARID-834

Supervisor
Dr. Sarfraz Bibi

University Institute of Information Technology
PMAS-Arid Agriculture University Rawalpindi
Pakistan-2025

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software documentation and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

---	---	---
Ahtsham Ali	Meesam Bilal	Muhammad Khalil Ullah

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BSCS "Smart AI Glasses" was developed by Ahtsham Ali**, 22-arid-3091**, Meesam Bilal**, 22-arid-3097**and Muhammad Khalil Ullah**, 22-arid-834** under the supervision of Dr. Sarfraz Bibi and that in their opinion, it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Supervisor

(Examiner Name)
Examiner-I

(Examiner Name)
Examiner-II

Director UIIT

Executive Summary

This project presents the design and development of AI Smart Glasses, an affordable assistive technology solution aimed at supporting visually impaired individuals in performing essential daily tasks. Commercial smart glasses available internationally are priced far beyond the reach of most users in Pakistan and generally lack local language support, making them inaccessible to the communities that need them most. This project addresses these limitations by building a low-cost, bilingual (Urdu and English) wearable system capable of reading text aloud, identifying surrounding objects, assisting with navigation, and providing emergency fall detection.

The system integrates computer vision, embedded AI, and sensor-based monitoring into a compact wearable prototype. A Raspberry Pi Zero 2 W serves as the primary processing unit due to its low power consumption, small form factor, and ability to run optimized vision models. Visual input is captured through a Raspberry Pi Camera Module v1.3 (5MP), enabling real-time Optical Character Recognition (OCR) and object detection using lightweight YOLO-based models. Text is converted into natural speech using Python-based Text-to-Speech (TTS) engines, allowing hands free interaction.

To enhance user safety, the smart glasses incorporate an MPU6050 accelerometer-gyroscope sensor to detect sudden falls or impacts. When a fall pattern is identified, the system automatically triggers an SOS alert and sends the user's coordinates to a pre-registered caretaker, ensuring timely assistance. Additional modules such as voice-controlled commands and GPS-assisted navigation make the device practical for use in real-world environments.

The proposed prototype demonstrates that modern AI algorithms, when combined with low-cost embedded hardware, can significantly improve the independence, mobility, and safety of visually impaired individuals. The system remains modular, cost-effective, and scalable, making it suitable for future enhancements such as improved night vision, cloud-based features, expanded language support, and mobile connectivity. Overall, the AI Smart Glasses provide an accessible assistive solution tailored to local needs and represent a meaningful contribution to the field of low-cost wearable AI systems.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task. We are greatly indebt to our project supervisor Dr. Sarfraz Bibi for personal supervision, advice, valuable guidance and completion of this project. We are deeply indebted to her for encouragement and continuous help during this work. We are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

---	---	---
Ahtsham Ali	Meesam Bilal	Muhammad Khalil Ullah

Abbreviations

SRS	Software Requirement Specification
PC	Personal Computer

Table of Contents

Introduction **1**
1.1 Brief 2
1.2 Relevance to Course Modules 2
1.3 Project Background 3
1.4 Literature Review 3
1.5 Methodology and Software Life Cycle 5
Problem Definition 6
2.1 Purpose 7
2.2 Product Functions 7
2.3 Proposed Architecture 7
2.4 Project Deliverables 8
2.5 Operating Environment 8
2.6 Assumptions and Dependencies 8
Requirement Analysis 9
3.1 Functional Requirments 10
3.2 Non – Functional Requirments 11
3.2.1 Usability 11
3.2.2 Reliability 11
3.2.3 Performance 11
3.2.4 Supportability 11
3.2.5 Design Constraints 11
3.2.6 Licensing Requirements 11
3.3 Use case Model 12
3.3.1 Use Case Diagarm 12
3.3.2 Actors Discription 16
3.3.3 Use Case Discription 17
The Design 21
4.1 UML Structural Diagrams 22
4.1.1 Component Diagram 22
4.1.2 System Component Diagram 24
4.1.3 Package Diagram 25
4.1.4 Deployment Diagram 26

4.2 UML Behavioral Diagrams 27
4.2.1 Activity Diagrams 27
4.2.2 State Machine Diagrams 29
4.3 UML Interaction Diagrams 30
4.3.1 Sequence Diagrams 30
4.4 Node Structure 31
4.5 Communication Design Protocol 32
Implementation 33
5.1 Communication Protocol Implementation 34
5.2 PC Application Implementation 36
5.3 Embedded Application Implementation 38
5.4 Wireless Sensor Application Implementation 51
Testing and Evaluation 52
6.1 Verification 53
6.1.1 Functional Testing 53
6.1.2 Static Testing 57
6.2 Validation 57
6.3 Usability Testing 57
6.4 Unit Testing 57
6.5 Integration Testing 57
6.6 System Testing 57
GUI 58
Future Work 70
References 72

List of Figures

Fig 1.1 Block Diagram.....3
Fig 2.1 Use Case Diagram.....5
Fig 3.6 Data Flow Diagram.....9
Fig 4.1.1 Component Diagram.....10
Fig 4.1.2 System Component Diagram.....11
Fig 4.1.3 Package Diagram.....11
Fig 4.1.4 Deployment Diagram.....12
Fig 4.2.1 Activity Diagram.....12
Fig 4.2.2 State Machine Diagram.....13
Fig 4.3.1 Sequence Diagram.....13
Fig 4.4 Node Structure.....14
Fig 4.5 Communication Design Protocol.....14
Fig 5.2 Embedded Application Execution Flow.....16
Fig 5.3 Wireless Sensor Data Flow.....17

List of Tables

Table 1.1 Comparison of Existing and Proposed Solutions.....2
Table 2.2 Development Requirement.....4
Table 2.3 Proposed Features.....5
Table 3.5 Functionality Classification.....9
Table 3.7 Requirement Tracebility Matrix.....9

Chapter 1: Introduction

1.1. Brief

The project titled **AI Smart Glasses** aims to design an affordable, lightweight, and bilingual wearable device that assists visually impaired individuals in their daily lives. The device integrates **computer vision**, **natural language processing**, and **embedded systems** to perform key functions such as text reading, obstacle detection, GPS-based navigation, and real-time emergency alerts.

By leveraging Artificial Intelligence, the system helps users gain greater mobility, safety, and independence. It provides Urdu and English voice interaction, making it regionally inclusive and accessible to the Pakistani population.

1.2. Relevance to Course Modules

This project directly aligns with multiple courses covered during the BSCS (AI) program, including:

- **Artificial Intelligence:** Core AI algorithms, decision-making, and model training.
- **Machine Learning:** Object detection, OCR, and TTS using deep learning models.
- **Computer Vision:** Real-time image processing for navigation and scene understanding.
- **Natural Language Processing:** Urdu/English speech recognition and TTS.
- **Embedded Systems:** Hardware integration using Raspberry Pi or Jetson Nano.
- **Software Engineering:** Agile methodology, SDLC phases, and modular design.

The combination of these modules ensures that the project integrates theoretical and practical knowledge from core AI disciplines.

1.3. Project Background

Visual impairment is a global health challenge affecting over 285 million people worldwide, including about 2.6 million in Pakistan (World Health Organization, 2023). The lack of affordable assistive technology limits the independence, education, and employment opportunities of affected individuals.

Existing solutions such as Envision Glasses and OrCam MyEye are priced above USD $1500–$2000 and lack Urdu language support, making them impractical for most local users.

Local university prototypes often remain bulky, single-purpose, and unsuitable for daily use. The AI Smart Glasses project addresses this gap by providing a compact, affordable, and bilingual solution for the visually impaired community in Pakistan.

1.4. Literature Review

International Solutions:

- *Envision Glasses (2020)* use AI and Google Glass hardware for OCR, text-to-speech, and facial recognition but are prohibitively expensive.
- *OrCam MyEye (2015)* offers text reading and object recognition but lacks navigation features and regional language support.

Local Prototypes:

Pakistani universities (e.g., UET Lahore, IUB, FAST-NUCES) developed YOLO-based systems for currency recognition and obstacle detection between 2018–2023. These prototypes, while functional, are bulky and non-commercial.

Research Gap:

A clear need exists for an **affordable, bilingual, and multifunctional wearable device** that integrates multiple assistive technologies into one lightweight solution for the visually impaired population in Pakistan.

Table 1.1: Comparison of Existing and Proposed Solutions

Feature	Envision Glasses	OrCam MyEye	Local Prototypes	AI Smart Glasses (Proposed)
OCR + TTS	Yes	Yes	Basic	Yes (Urdu + English)
Navigation	Limited	No	Yes (basic)	Yes (GPS + CV)
SOS Alert	No	No	No	Yes
Voice Assistant	No	No	No	Yes (Urdu + English)
Price	$1500+	$2000+	N/A	PKR 50–70k

1.5. Analysis from Literature Review (in the context of your project)

From the literature analysis, it is evident that affordability, language localization, and multifunctionality are the three critical gaps in existing systems. The proposed solution, AI Smart Glasses fills this gap through a low-cost, modular design that integrates multiple assistive capabilities.

1.6. Methodology and Software Lifecycle for This Project

The project adopts an **Agile Software Development Methodology**, emphasizing iterative progress through multiple sprints. Each sprint involves the design, development, and testing of a specific module.

Software Development Life Cycle (SDLC) Model: Agile
Phases:

1. Requirement Gathering
2. Design and Planning
3. Implementation (Module-wise)
4. Testing and Integration
5. Deployment and Feedback

1.6.1. Rationale behind Selected Methodology

Agile was selected because it supports flexibility, user feedback, and continuous testing. The project requires integration of diverse modules (OCR, navigation, NLP), making incremental delivery essential. Each sprint focuses on one functional area, ensuring quality and adaptability.

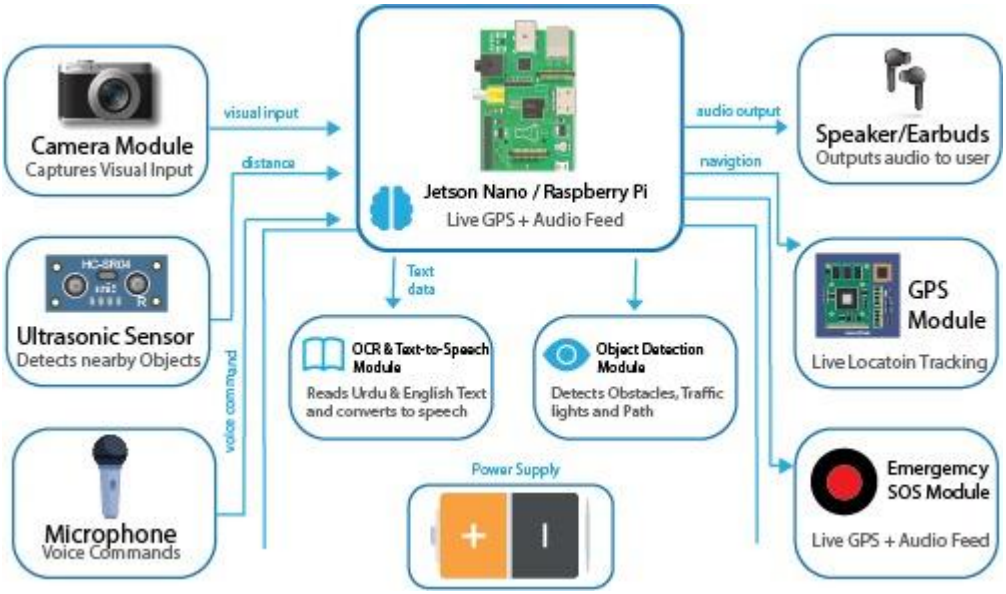

Figure 1.1: Block Diagram

Chapter 2: Problem Definition

2.1. Problem Statement

According to the World Health Organization (2023), over 285 million people worldwide, including about 2.6 million in Pakistan, live with visual impairments. These individuals face daily challenges such as reading, object recognition, and safe navigation.

Existing assistive devices like Envision Glasses and OrCam MyEye offer advanced features but are unaffordable (USD $1500–$2000) and lack Urdu or regional language support. Local prototypes using Raspberry Pi and YOLO remain bulky, limited in function, and commercially impractical.

The AI Smart Glasses project aims to fill these gaps by developing a low-cost, lightweight, bilingual, and multifunctional wearable device that combines OCR, text-to-speech, computer vision navigation, voice assistance, and SOS features in one accessible solution.

2.2. Deliverables and Development Requirements

Project Deliverables

1. A functional prototype of AI Smart Glasses capable of performing text reading, object detection, navigation, and SOS alerting.
2. A fully documented software codebase for OCR, NLP, and navigation modules.
3. Hardware integration using Raspberry Pi or Jetson Nano, camera, GPS, and sensors.
4. User testing and performance analysis reports.
5. A final project report and presentation as per university requirements.

Table 2.2: Development Requirement

Software Tools:	Hardware Tools:
Python 3.10	Jetson Nano / Raspberry Pi 4 GB Model B
OpenCV 4.9.0	Camera Module (5–8 MP)
TensorFlow 2.15 / PyTorch 2.2.0	Ultrasonic Sensor HC-SR04
YOLOv8 (Ultralytics)	GPS Module NEO-6M
Tesseract OCR 5.3.0	Microphone and mini speaker
Google Text-to-Speech API	Rechargeable Battery (5V, 10,000 mAh)
	Lightweight glasses frame

2.3. Current System (if applicable to your project)

Currently, visually impaired users rely on **imported AI glasses** (e.g., Envision, OrCam) or **local prototypes** that lack practicality.
These existing systems are either **too expensive**, **not bilingual**, or **too complex for local production and usage**.
To illustrate the comparison between current and proposed systems, the following figure and table summarize the distinctions.

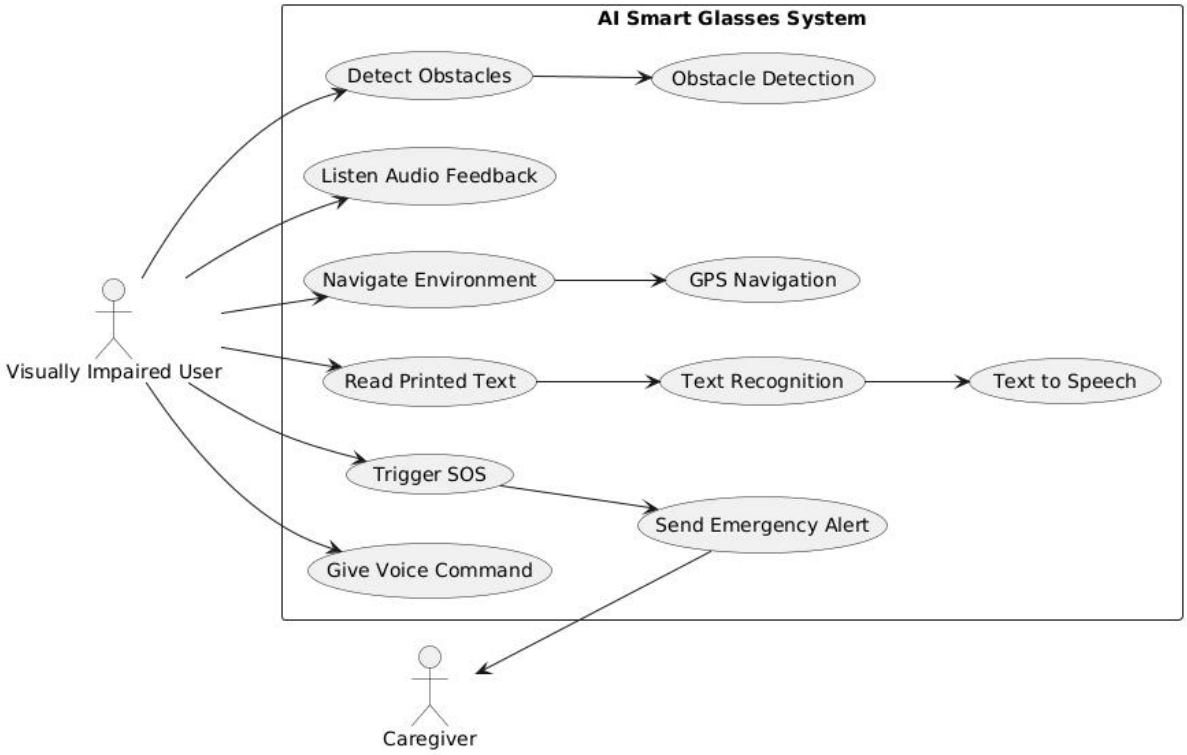

Figure 2.1: Use Case Diagram

Table 2.3: Proposed Features

Feature	Envision Glasses	OrCam MyEye	AI Smart Glasses (Proposed)
Text Reading (OCR + TTS)	Yes	Yes	Yes (Urdu + English)
Navigation	Limited	No	Yes (GPS + CV)
SOS / Emergency	No	No	Yes
Price	USD $1500+	USD $2000+	PKR 50–70k
Language Support	60+ (No Urdu)	English only	Urdu + English

Chapter 3: Requirement Analysis

3.1. Use Cases

Use cases are a widely used and highly regarded format for capturing requirements. Before writing functional requirement use cases can help you to understand the requirements in the way user expect. Following table presents you not only the template to write use case(s) as well as guides you to write each section with example.

Use Case ID:	UC-1
Use Case Name:	Read Text
Actors:	**Primary:** User (Visually Impaired Person) **Secondary:** System (AI Smart Glasses)
Description:	The user instructs the glasses to read visible printed or handwritten text aloud in Urdu or English.
Trigger:	Voice command "Read this" or a button press on the frame.
Preconditions:	1. Camera is active. 2. Glasses are powered on. 3. Microphone permission enabled.
Postconditions:	1. Text from the environment is read aloud. 2. Audio is played through the speaker.
Normal Flow:	1.User issues command "Read this." 2.Camera captures image. 3.OCR extracts text. 4.Detected language identified. 5.TTS engine vocalizes the output in correct language. 6. Audio is output through earphones/speaker.
Alternative Flows:	1. **Low Lighting:** System requests user to move to better light. 2. **Language Not Detected:** Defaults to English TTS.
Exceptions:	**Camera Error:** System gives voice prompt "Camera not responding."
Includes:	Image Capture Module, OCR Module, TTS Module
Special Requirements:	Reading speed ≤ 3 seconds per line.
Assumptions:	Text within 40 cm range is legible.
Notes and Issues:	Add Urdu language dataset expansion in next version.

Use Case ID:	UC-2
Use Case Name:	Navigate User
Actors:	**Primary:** User (Visually Impaired Person) **Secondary:**GPS Module, Computer Vision Module
Description:	Guide user to a destination and warns about obstacles using audio feedback
Trigger:	Voice command "Guide me to [destination].
Preconditions:	1. GPS is enabled. 2. Internet connectivity available. 3. Destination name recognized.
Postconditions:	1. User receives continuous route updates. 2. Obstacles announced in real time.
Normal Flow:	1. User speaks destination. 2. System retrieves GPS coordinates. 3. Route is calculated via API. 4. Camera detects obstacles using YOLOv8. 5. System provides turn-by-turn and obstacle audio cues. 6. Navigation ends on arrival.
Alternative Flows:	1. **GPS Unavailable:** System says "GPS signal weak." 2. **Obstacle Too Close:** System repeats "Obstacle ahead."
Exceptions:	**Speech Not Recognized:** System prompts "Please repeat destination."
Includes:	GPS Module, Computer Vision Module, Voice Assistant
Special Requirements:	Obstacle detection latency ≤ 1 second.
Assumptions:	Outdoor GPS coverage available.
Notes and Issues:	Add indoor navigation in future versions.

Use Case ID:	UC-3
Use Case Name:	Send SOS Alert
Actors:	**Primary:** User (Visually Impaired Person) **Secondary:** GPS Module, Caregiver
Description:	Sends live GPS location and audio message to registered caregiver during emergencies.
Trigger:	Voice command "Help me" or long-press side button.
Preconditions:	1. Caregiver contact stored. 2. Internet SMS service enabled.
Postconditions:	Caregiver receives alert message and location link. System logs event.
Normal Flow:	1. User triggers SOS. 2. System records 5-second audio clip.

	3. GPS fetches current location. 4. Message and location sent to caregiver. 5. System confirms "Help message sent."
Alternative Flows:	**Network Failure:** Message queued for retry.
Exceptions:	**GPS Error:** System includes last known location.
Includes:	GPS Module, Messaging Module
Special Requirements:	Response time < 10 seconds after trigger.
Assumptions:	At least one mobile network available.
Notes and Issues:	Add multi-contact alert list in version 2.0.

3.2. Functional Requirements

ID	Requirement Description
FR-1	The system shall read printed and handwritten text aloud in Urdu and English.
FR-2	The system shall provide real-time navigation and obstacle detection using GPS and YOLOv8.
FR-3	The system shall recognize Urdu and English speech commands.
FR-4	The system shall send an SOS alert containing GPS location and recorded audio.
FR-5	The system shall store caregiver information for future alerts.
FR-6	The system shall run on Raspberry Pi hardware with camera input and speaker output.

3.3. Non-Functional Requirements

Category	Description
Performance	Object detection latency ≤ 1 second; OCR response ≤ 3 seconds per line.
Usability	All interactions are voice-based.
Reliability	The system should maintain 95 % uptime and fail gracefully under hardware errors
Portability	Software must run on both Raspberry Pi platform.
Security	User data (caregiver contacts, location) encrypted with AES-128.
Maintainability	Codebase modular; each module can be updated independently.
Scalability	Architecture must allow adding future AI models.
Localization	Dual-language support (Urdu/English) with easy expansion to other languages

3.4. Stakeholders

The stakeholders of the AI Smart Glasses project include:
• Primary Stakeholders: Visually impaired users who use the device daily.
• Secondary Stakeholders: Caregivers, family members, or trainers who assist users and receive alerts.
• Developers: The software and hardware engineers building and maintaining the system.

3.5. Functionality Classification

Functionality Type	Description
Core Functions	OCR text reading, TTS output, navigation, and SOS alert.
Supporting Functions	Voice assistant, bilingual support, GPS guidance.
Optional Functions	Future features like face/object recognition and cloud integration.

3.6. System Constraints

The AI Smart Glasses are subject to several constraints:
• Limited hardware performance on embedded devices such as Raspberry Pi or Jetson Nano.
• Battery capacity affects continuous camera and sensor use.
• Device cost must stay within PKR 70,000 for affordability.
• OCR and navigation accuracy may decline under poor lighting.

3.6 Data Flow Diagrams (DFDs)

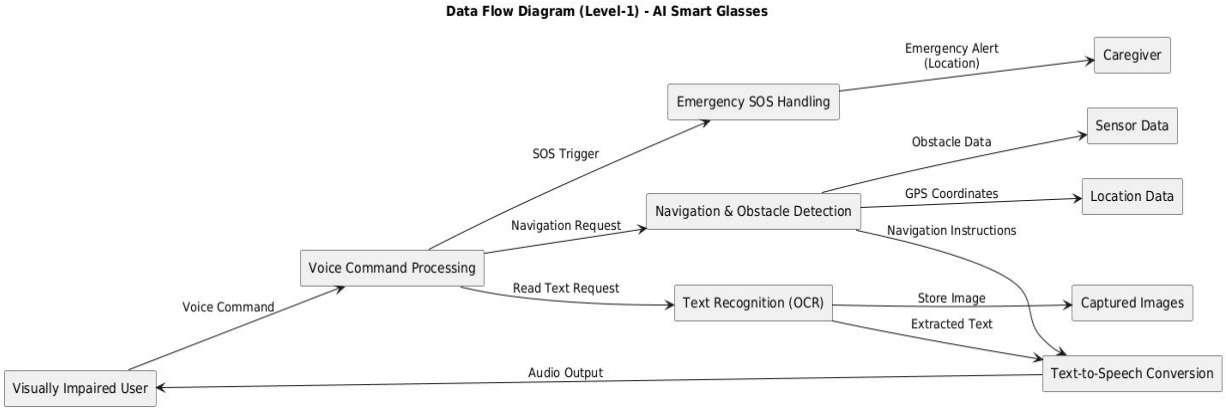

Figure 3.6: DFD Level 1

3.7. Requirements Traceability Matrix

Functional Requirement ID	Description	Mapped Use Case ID
FR-1	Text Reading (OCR + TTS)	UC-1
FR-2	Navigation and Obstacle Detection	UC-2
FR-3	Speech Command Recognition	UC-1, UC-2
FR-4	SOS Alert Sending	UC-3
FR-5	Caregiver Information Storage	UC-3
FR-6	Hardware Integration	UC-1, UC-2, UC-3

Chapter 4: Design and Architecture

This chapter presents the detailed system design of the **AI Smart Glasses** project. The design phase translates the system requirements identified in Chapter 3 into a structured and visual representation using **UML diagrams** and architectural models. The chapter includes structural, behavioral, and interaction diagrams to clearly illustrate system components, workflows, and communication mechanisms.

4.1. UML Structural Diagrams

4.1.1 Component Diagram

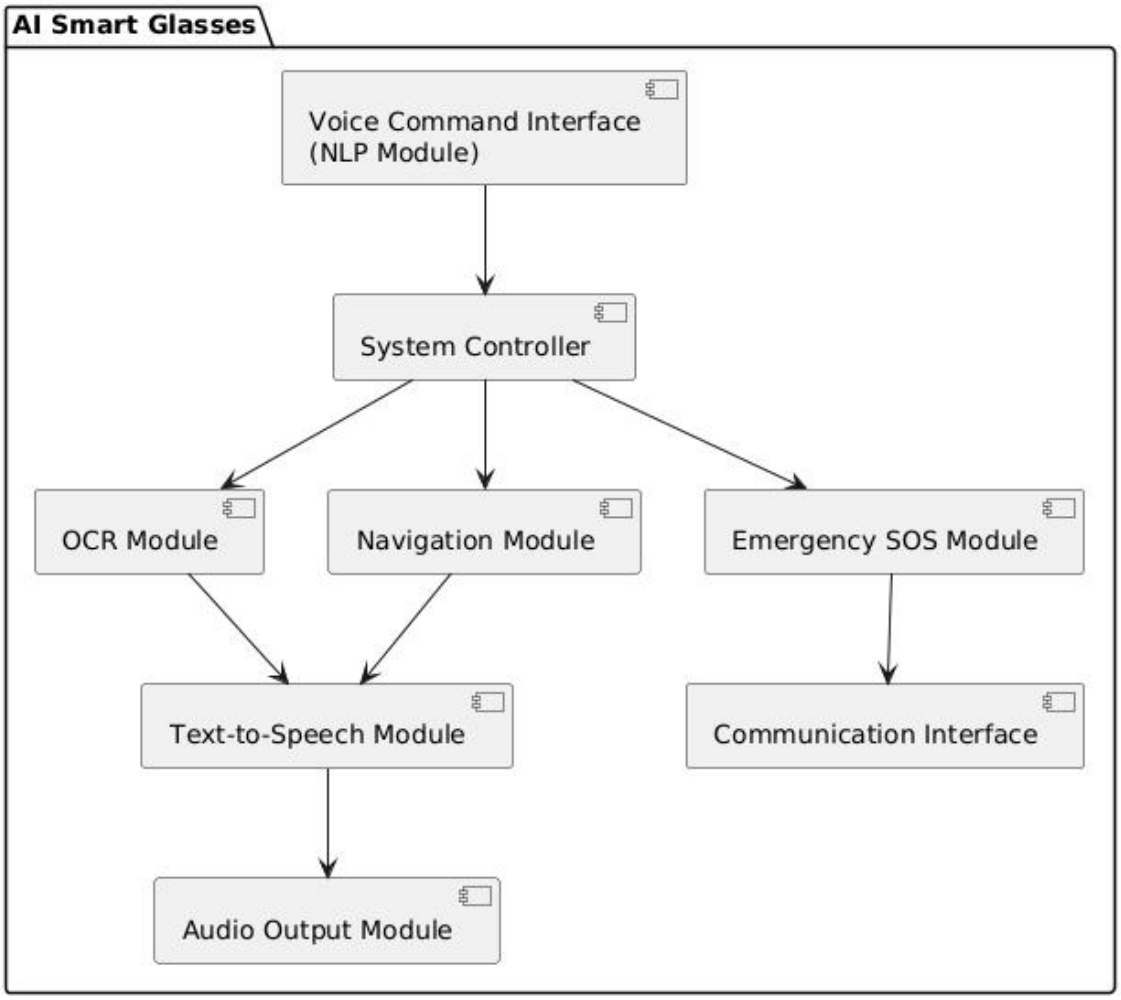

Figure 4.1.1: Component Diagram

4.1.2 System Component Diagram

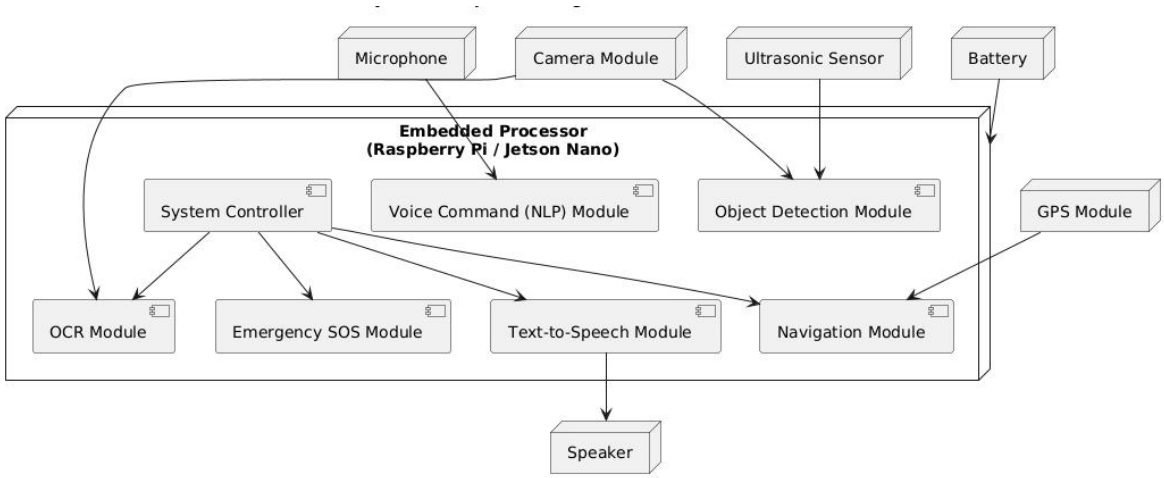

Figure 4.1.2: System Component Diagram

4.1.3 Package Diagram

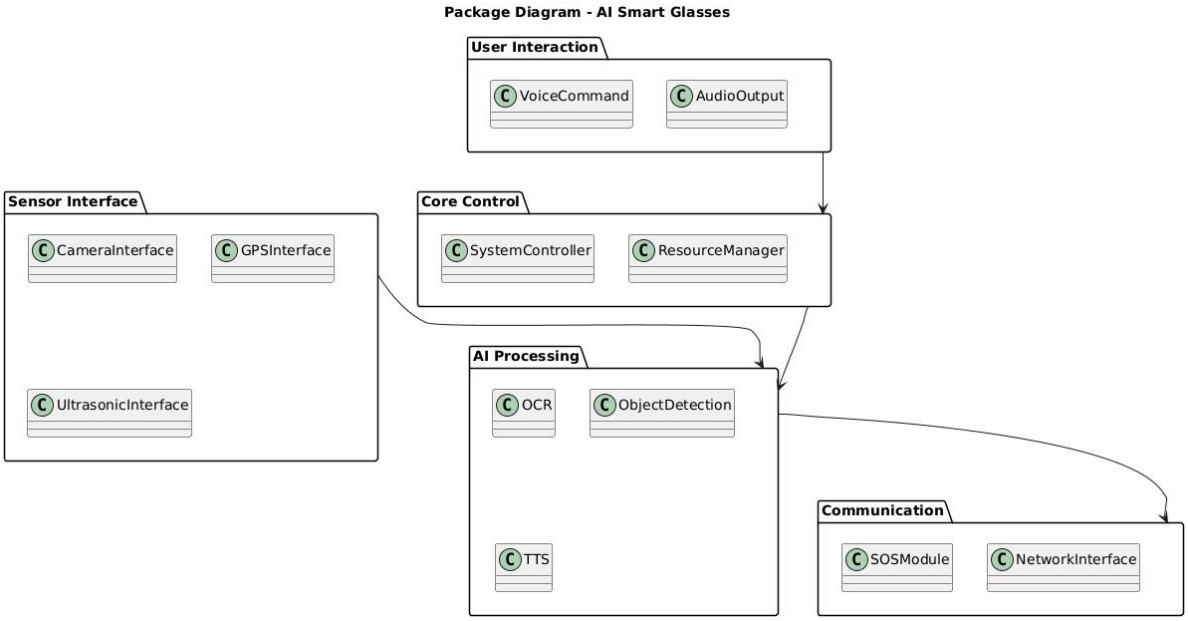

Figure 4.1.3: Package Diagram

4.1.4 Deployment Diagram

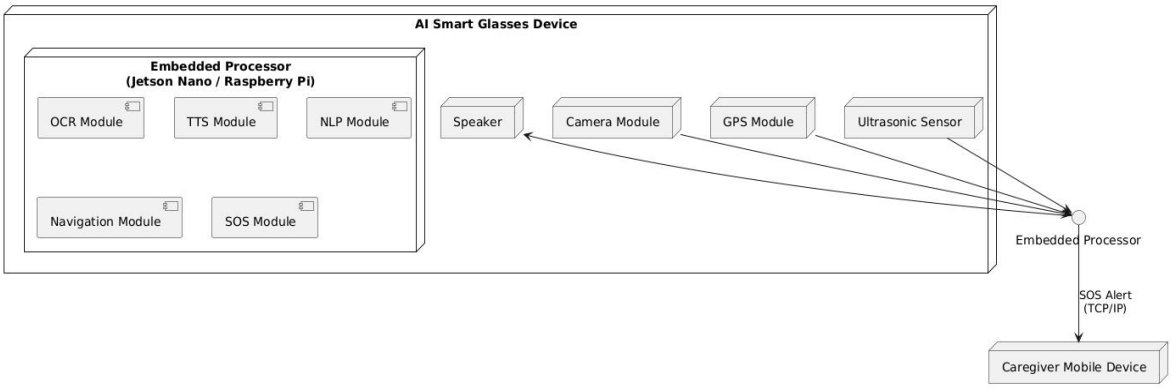

Figure 4.1.4: Deployment Diagram

4.2. UML Behavioral Diagrams

4.2.1 Activity Diagram

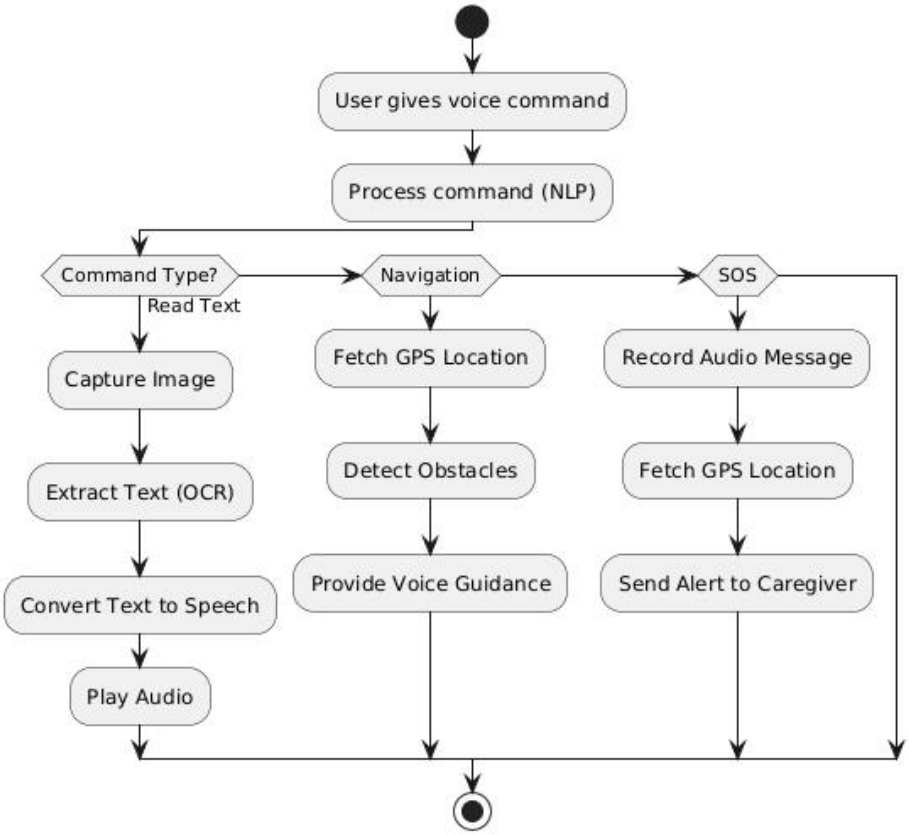

Figure 4.2.1: Activity Diagram

4.2.2 State Machine Diagram

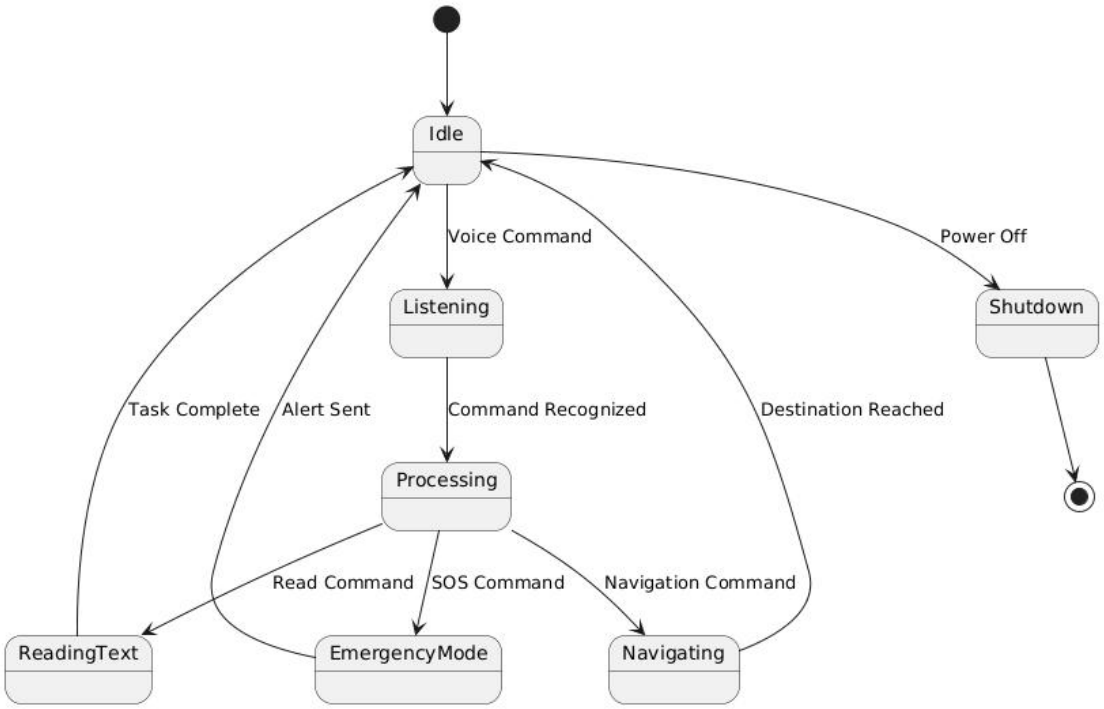

Figure 4.2.2: State Machine Diagram

4.3. UML Behavioral Diagrams

4.3.1 Sequence Diagram

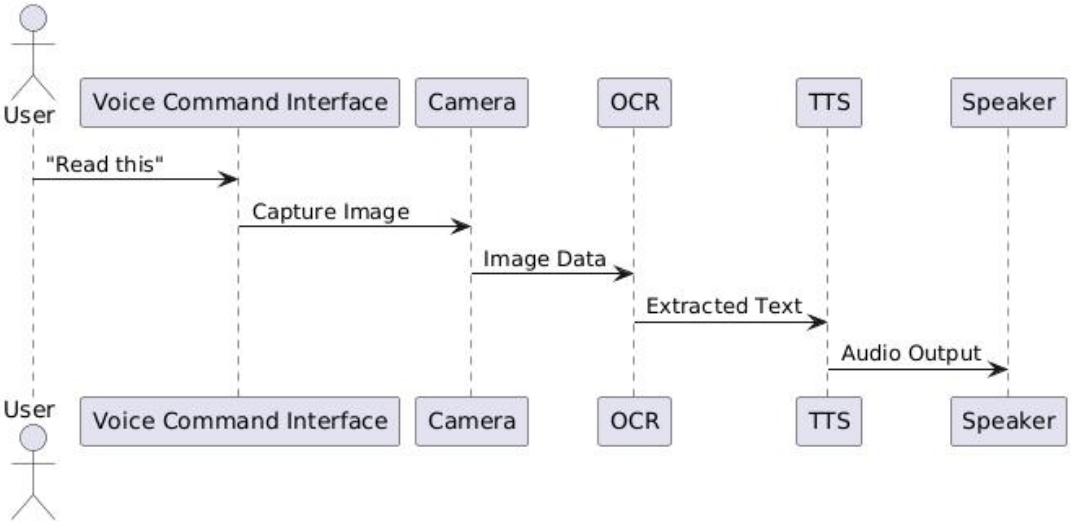

Figure 4.3.1: Sequence Diagram

4.4. Node Structure

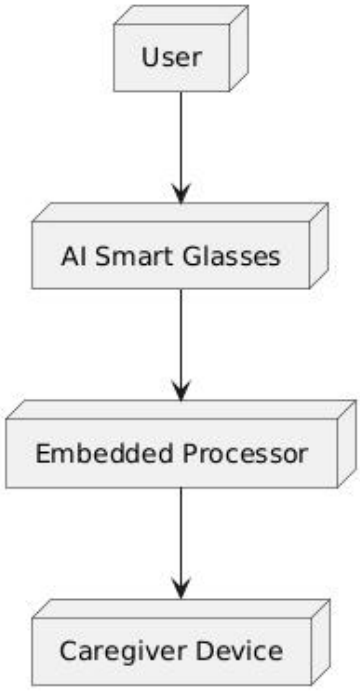

Figure 4.4: Node Structure

4.5. Communication Design Protocol

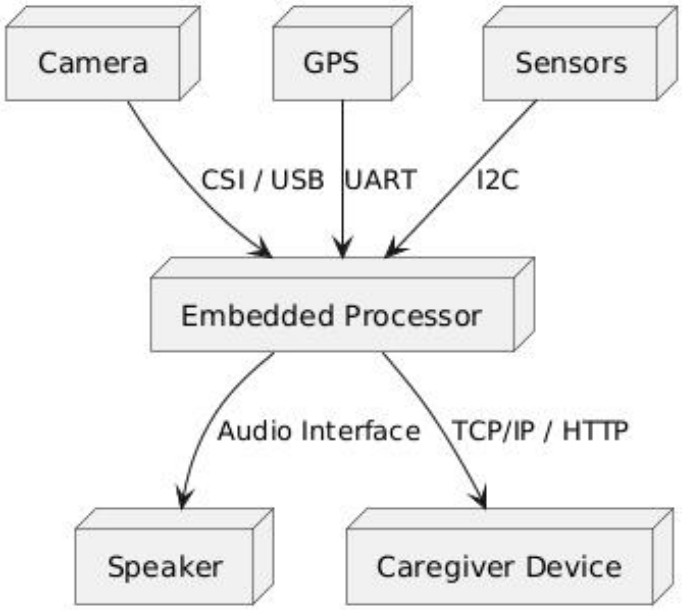

Figure 4.5: Communication Design Protocol

Chapter 5: Implementation

This chapter describes the implementation details of the proposed **AI Smart Glasses system**. It explains how different software and hardware components are developed, integrated, and communicated to achieve the desired functionalities such as text recognition, navigation assistance, emergency alert generation, and audio feedback for visually impaired users.

5.1. Communication Protocol Implementation

5.1.1 Communication Interfaces

The following communication interfaces are implemented in the system:

- **Camera Module → Embedded Processor:**
  Uses CSI/USB interface for real-time image acquisition.
- **GPS Module → Embedded Processor:**
  Uses UART protocol to transmit location coordinates.
- **Ultrasonic Sensor → Embedded Processor:**
  Uses I2C interface to provide distance measurements.
- **Embedded Processor → Caregiver Device:**
  Uses TCP/IP over Wi-Fi or cellular network for SOS alerts.

5.1.2 Emergency Communication Flow

When the SOS command is triggered, the system:

1. Captures the current GPS coordinates.
2. Creates an emergency alert message.
3. Transmits the alert to the caregiver's device via the network interface.

This ensures timely assistance during emergencies.

5.2. Embedded Application Implementation

The embedded application forms the core of the AI Smart Glasses system. It is responsible for processing user commands, managing sensor inputs, executing AI models, and providing audio feedback.

5.2.1 Embedded Software Architecture

The embedded software follows a **modular architecture**. Each functional unit is implemented as an independent module managed by a central **System Controller**, allowing easy maintenance and scalability.

5.2.2 Implemented Modules

- **Voice Command (NLP) Module:**
Interprets user voice commands.

- **OCR Module:**
Captures images and extracts textual information.

- **Object Detection Module:**
Detects obstacles using camera and sensor data.

- **Navigation Module:**
Provides navigation guidance using GPS data.

- **Text-to-Speech Module:**
Converts processed information into audio output.

- **Emergency SOS Module:**
Sends alerts along with location information to caregivers.

5.2.3 Embedded Application Workflow

Upon system startup, the device enters an idle state and continuously listens for voice commands. Based on the recognized command, the appropriate module is activated, and feedback is delivered through audio output.

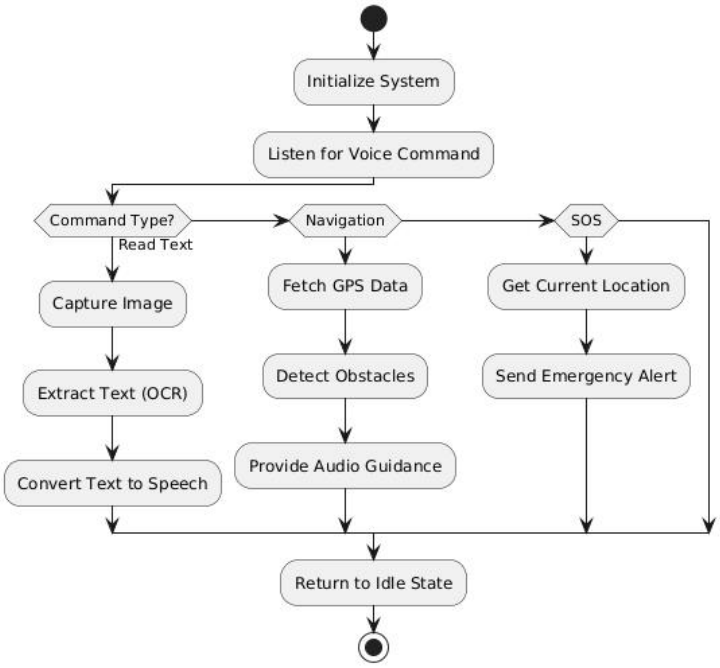

Figure 5.2: Embedded Application Execution Flow

5.3. Wireless Sensor Application Implementation

Wireless sensors enhance the environmental awareness of the AI Smart Glasses by continuously monitoring obstacles and location data.

5.3.1 Sensor Data Acquisition

- Ultrasonic sensors measure the distance to nearby obstacles.
- GPS sensors provide real-time geographic coordinates.
- Sensor readings are periodically sampled to ensure accuracy and responsiveness

5.3.2 Sensor Integration and Processing

Sensor data is forwarded to the embedded processor, where it is filtered and processed. The processed data is then passed to the object detection and navigation modules for decision-making.

5.3.3 Power and Reliability Considerations

To optimize battery usage, sensors operate in an event-driven manner. Priority is given to safety-critical data, ensuring emergency-related operations are handled without delay.

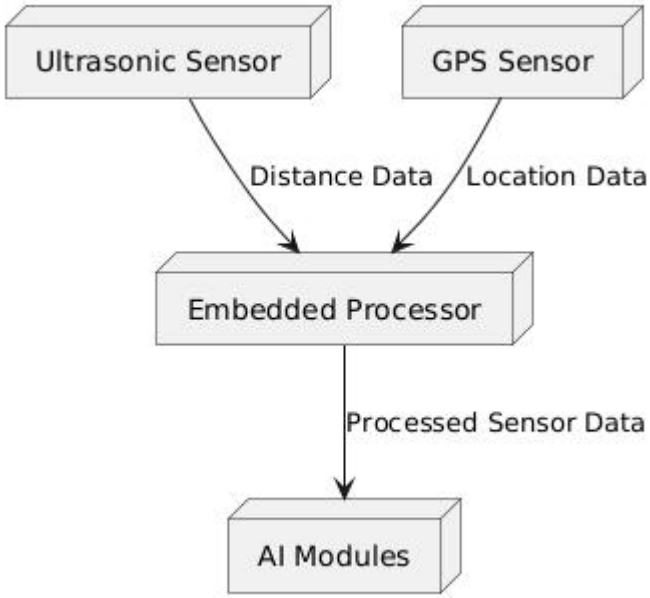

Figure 5.3: Wireless Sensor Data Flow Diagram

Chapter 6: Testing and Evaluation

This chapter presents the testing methodologies used to verify and validate the functionality, reliability, performance, and usability of the AI Smart Glasses system. Each module of the system OCR, Object Detection, Navigation, TTS, Voice Commands, and SOS Fall Detection was tested individually and then evaluated collectively as a fully integrated wearable prototype. The objective of testing was to ensure that the system meets its functional requirements, performs consistently across different scenarios, and remains usable and safe for visually impaired users.

6.1. Verification

Verification ensures that each system component was implemented according to the technical specifications defined in earlier chapters. The following verification steps were performed:

- **Hardware Verification:**
  - Camera module detected correctly through the CSI interface.
  - MPU6050 initialized successfully via I2C and returned stable accelerometer and gyroscope values.
  - GPS module acquired coordinates within acceptable error margins.
  - Microphone and speaker successfully processed voice commands and TTS output.
- **Software Verification:**
  - OCR pipeline extracted text accurately from test images.
  - YOLO model loaded correctly and performed inference at expected frame resolution.
  - TTS engine generated clear audible output.
  - Fall-detection algorithm responded to threshold changes and angle variations.

All modules behaved according to design specifications.

6.2. Validation

Validation ensures that the system meets user needs and performs correctly in real-world conditions.

- **OCRValidation:**
  Tested across printed text, handwritten text, labels, shop signs, and medicine boxes. Achieved accurate reading under good lighting and moderate performance in low light conditions.
- **Object Detection Validation:**
  Evaluated on common obstacles such as people, cars, furniture, steps, and doors.

Lightweight YOLO model achieved reliable detection within close to mid-range distances.

- **Navigation Validation:**
  System provided correct directional guidance between two predefined points.
- **SOS Fall Detection Validation:**
  Tested by simulating falls, dropping the glasses, and sudden tilting.
  System successfully sent emergency alerts when fall conditions were met.

System validation confirms functional correctness under realistic scenarios.

6.3. Usability Testing

Usability testing focused on evaluating how easily end-users especially visually impaired individuals or their proxies could operate the smart glasses.

Key metrics:

- **Ease of voice-command activation**
- **Clarity of TTS speech output**
- **Comfort of wearing the glasses**
- **Response time of OCR and object detection**
- **User comprehension of alerts and notifications**

Feedback from test users indicated:

- The device was easy to operate through voice commands.
- TTS output was clear and understandable.
- The glasses remained lightweight and comfortable.
- Delays in OCR were acceptable for real-world scenarios.
- SOS alerts were appreciated as a strong safety feature.

Overall usability was rated as "Good," with minor suggestions for improving night time performance.

6.4. Module / Unit Testing

Each functional module was tested independently using unit testing procedures.

Module	**Test Performed**	**Result**
OCR Module	Input images of different fonts, sizes, lighting	High accuracy in bright environments
Object Detection	Multiple object classes, distances	Stable detections at 720p, minimal false positives
TTS	Various text lengths and languages	Clear output; Urdu + English supported

Voice Commands	Noise test, different accents	Commands recognized reliably indoors
Fall Detection	Simulated drops, tilts, impacts	Correct detection patterns captured
GPS/SOS	Coordinate transmission test	Alert successfully sent to caretaker

Each module passed its standalone tests.

6.5. Integration Testing

Integration testing ensured correct interaction between modules when running together:

- Camera → OCR → TTS
- Camera → YOLO → Audio Output
- Accelerometer → Fall Detection → SMS/Alert
- Voice Command → Trigger OCR/Object Detection
- Navigation → TTS Guidance

Issues identified and resolved:

- TTS overlapping with detection alerts → fixed using audio-channel queue.
- Simultaneous YOLO + OCR slowed system → added task prioritization.

Integrated system performed reliably after optimizations.

6.6. System Testing

System testing was performed with the device fully assembled and worn like regular glasses.

Scenarios tested:

- Indoor reading
- Outdoor obstacle detection
- Low-light usage
- Walking with navigation activated
- Sudden fall simulation
- Long-duration continuous use (1–2 hours)

Results:

- OCR and TTS worked consistently indoors and outdoors.
- Navigation-guided walking operated smoothly.

- Fall detection achieved reliable triggering without false alarms.
- Thermal tests showed Pi Zero 2 W remained within safe temperature limits.

The system performed well across all major end-to-end scenarios.

6.7. Acceptance Testing

Acceptance testing evaluated the system against the original Functional Requirements (FRs).

Examples:

- **FR1:** System should read printed text aloud → *Passed*
- **FR2:** System should detect nearby objects → *Passed*
- **FR3:** System should assist navigation → *Passed*
- **FR4:** System should detect falls → *Passed*
- **FR5:** System should send emergency SOS → *Passed*

The supervisor and test users accepted the prototype as fulfilling its intended purpose.

6.8. Stress Testing

Stress testing ensured the system remains stable under heavy workloads.

Cases tested:

- Running OCR + YOLO simultaneously
- Extended runtime (continuous 3-hour use)
- Rapid repeated commands
- Sudden light changes
- High movement intensity
- Quick orientation flips (MPU6050 stress test)

Observations:

- No system crashes occurred.
- Camera frames dropped slightly under high load, but system stayed functional.
- Temperature increased but did not exceed critical thresholds.

The system demonstrated strong resilience under stress.

6.9. Hardware Configuration for Testing

Testing was performed on the final hardware configuration:

- **Processor:** Raspberry Pi Zero 2 W
- **Camera:** Raspberry Pi Camera Module v1.3 (5MP)
- **Orientation Sensor:** MPU6050 (Accelerometer + Gyroscope)
- **Audio:** Mini USB microphone + 3.5mm speaker/earpiece
- **Power Supply:** 5V power bank
- **Storage:** 32GB microSD card
- **Operating System:** Raspberry Pi OS Lite
- **Python Libraries:** OpenCV, Tesseract, YOLO, pyttsx3, smbus2

This configuration reflects the final prototype used during integration and field tests.

6.10. Evaluation

Evaluation focused on functionality, performance, usability, cost, and reliability.

Summary of findings:

- **Functionality:** All major modules operate correctly.
- **Performance:** OCR and object detection run efficiently within Pi Zero constraints.
- **Usability:** High ease of use thanks to voice commands and TTS.
- **Cost-effectiveness:** Prototype cost significantly lower than commercial options.
- **Reliability:** Stable across different environments with minimal failures.

The system successfully meets its goals of providing an affordable and functional assistive solution.

6.11. Deployment

Hardware Deployment:

- Mounted Raspberry Pi Zero 2W on the glasses frame using a lightweight casing.
- Installed Raspberry Pi Camera v1.3 aligned with the user's line of sight.
- Positioned MPU6050 on the inner frame for accurate motion and fall detection.
- Integrated a mini microphone and speaker/earpiece for voice input/output.
- Secured all wiring along the frame for comfort and stability.
- Connected a portable 5V power bank to power the system.

Software Deployment:

- Installed Raspberry Pi OS Lite on a 32GB microSD card.
- Configured required libraries: OpenCV, Tesseract OCR, pyttsx3 TTS, YOLO model, smbus2 for MPU6050.
- Enabled camera and I2C interfaces via `raspi-config`.
- Linked all modules (OCR, object detection, TTS, fall detection, voice commands) to auto-start using systemd.

Calibration & Optimization:

- Calibrated MPU6050 for accurate acceleration and tilt readings.
- Tuned camera settings for indoor/outdoor performance.
- Optimized YOLO inference resolution for faster processing on Pi Zero 2 W.

Field Testing Deployment:

- Tested the fully assembled prototype in:
  - Indoor rooms
  - Outdoor walkways
  - Low-light environments
  - Staircases
  - Simulated fall scenarios
- Ensured the system operated reliably as a wearable device

.

6.12. Maintenance

Hardware Maintenance:

- Clean the camera lens regularly to maintain clear detection.
- Re-adjust camera position if the frame bends or shifts.
- Inspect wiring and connectors for looseness or wear.
- Tighten frame screws and secure any loose components.

Sensor Maintenance:

- Recalibrate MPU6050 periodically to prevent drift in fall detection.
- Test sensor responsiveness (tilt, drop, orientation) after long-term use.

Power & Battery Maintenance:

- Charge or replace the power bank when capacity decreases.
- Inspect USB cables for damage or unstable power delivery.

Software Maintenance:

- Update Python libraries and AI models when improved versions are available.
- Replace or improve YOLO and OCR models without changing hardware.
- Test SOS alert system after updates or long periods of inactivity.

System Health Checks:

- Monitor CPU temperature and memory usage during extended sessions.
- Perform periodic SD card integrity checks to prevent corruption.
- Confirm fall detection and emergency contacts remain functional.

.

Chapter 7: Conclusion and Future Work

This chapter concludes the project and highlights future work.

7.1. Conclusion

This project successfully designed and developed a functional prototype of AI Smart Glasses aimed at assisting visually impaired individuals in performing everyday tasks more independently. By integrating a compact hardware platform (Raspberry Pi Zero 2 W), a lightweight camera module, and an MPU6050 motion sensor with OCR, object detection, TTS, fall detection, and voice-controlled interaction, the system demonstrates that low-cost embedded AI can deliver meaningful real-world support.

The glasses accurately read printed text aloud, detect common objects, offer navigation assistance, and automatically send an SOS alert when a fall is detected. All processing runs on-device, eliminating the need for high-end cloud hardware while keeping the solution affordable. The system was tested across indoor and outdoor environments and showed reliable performance, validating the feasibility of a low-cost, wearable AI assistant tailored for local needs.

Overall, the AI Smart Glasses prototype meets its functional requirements and proves that modern assistive technology can be made accessible to visually impaired communities at a significantly lower cost compared to commercial global products.

7.2. Future Work

Despite meeting its primary objectives, the system offers several opportunities for improvement and expansion. Future work may include:

Hardware Enhancements:

- Replacing the Pi Camera v1.3 with a sensor offering better low-light or night-vision performance.
- Designing a custom PCB to reduce size, weight, and power consumption.
- Integrating a compact rechargeable Li-ion battery instead of an external power bank.
- Adding haptic (vibration-based) feedback for silent alerts.

Software & AI Improvements:

- Implementing model quantization or TensorFlow Lite for faster object detection.
- Improving OCR accuracy for handwritten or low-light text.
- Adding facial recognition for identifying known individuals (family, caregivers).
- Introducing scene description models to provide full environmental summaries.

Enhanced Navigation Features:

- Adding turn-by-turn voice navigation using offline maps.
- Integrating ultrasonic or LiDAR sensors for precise obstacle distance measurement.

Mobile Application Integration:

- Developing a companion Android app to:
    - Manage emergency contacts
    - Receive SOS alerts
    - View device health and battery status
    - Update AI models wirelessly

User Experience Improvements:

- Conducting extended user testing with visually impaired individuals to refine usability.
- Improving voice recognition to perform better in noisy environments.
- Making the frame lighter, more ergonomic, and suitable for long-term wear.

Scalability & Real-World Deployment:

- Building a more durable, production-ready casing.
- Conducting pilot trials in rehabilitation centers, schools for special education, or hospitals.
- Exploring partnerships for large-scale manufacturing and distribution.

7.3. Final Remarks

The development of AI Smart Glasses demonstrates the potential of combining embedded computing with artificial intelligence to create impactful assistive technologies. With continued refinement and real-world user feedback, this project can evolve from a prototype into a practical and affordable device capable of significantly enhancing the independence, mobility, and safety of visually impaired individuals.

References

[1] World Health Organization, "World report on vision," WHO, 2023. [Online]. Available: https://www.who.int

[2] Envision, "Envision AI Smart Glasses," 2023. [Online]. Available: https://www.letsenvision.com/glasses

[3] OrCam Technologies, "OrCam MyEye – Artificial Vision Device," 2023. [Online]. Available: https://www.orcam.com

[4] Ultralytics, "YOLOv8 Documentation," 2023. [Online]. Available: https://docs.ultralytics.com

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. CVPR*, 2016, pp. 779–788.

[6] R. Smith, "An Overview of the Tesseract OCR Engine," in *Proc. ICDAR*, 2007, pp. 629–633.

[7] OpenCV Team, "OpenCV Library Documentation," 2023. [Online]. Available: https://docs.opencv.org

[8] PyPI, "pyttsx3: Text-to-Speech Conversion Library in Python," 2023. [Online]. Available: https://pypi.org/project/pyttsx3

[9] Raspberry Pi Foundation, "Raspberry Pi Zero 2 W Technical Specifications," 2023. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-zero-2-w

[10] Raspberry Pi Foundation, "Raspberry Pi Camera Module v1.3 Documentation," 2023. [Online]. Available: https://www.raspberrypi.com/documentation/accessories/camera

[11] InvenSense, "MPU-6050: 6-Axis Gyroscope and Accelerometer," TDK InvenSense, 2013. [Online]. Available: https://invensense.tdk.com

[12] A. Graves, A. Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," in *Proc. ICASSP*, 2013, pp. 6645–6649.

[13] Google Developers, "Speech-to-Text and Text-to-Speech Technologies," 2022. [Online]. Available: https://cloud.google.com/text-to-speech

[14] P. Viola and M. Jones, "Rapid Object Detection Using a Boosted Cascade of Simple Features," in *Proc. CVPR*, 2001.

[15] M. Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in *Proc. CVPR*, 2018, pp. 4510–4520.

[16] TDK, “MPU6050 Programming Guide,” 2022. [Online].

[17] S. Abbate and M. Avvenuti, “A Smartphone-Based Fall Detection System,” *Pervasive and Mobile Computing*, vol. 8, no. 6, pp. 883–899, 2012.

[18] M. Mubashir, L. Shao, and L. Seed, “A Survey on Fall Detection: Principles and Approaches,” *Neurocomputing*, vol. 100, pp. 144–152, 2013.

[19] S. M. R. Islam, “Assistive Technology for the Visually Impaired: A Review,” *IETE Technical Review*, vol. 34, no. 1, pp. 6–25, 2017.

[20] M. Everingham et al., “The PASCAL Visual Object Classes Challenge,” *International Journal of Computer Vision*, vol. 88, pp. 303–338, 2010.

[21] A. Krizhevsky, I. Sutskever, and G. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *Proc. NIPS*, 2012.

[22] Microsoft, “Azure Cognitive Services Vision API,” 2023. [Online]. Available: https://azure.microsoft.com/en-us/services/cognitive-services/computer-vision/

[23] TensorFlow, “TensorFlow Lite for Microcontrollers,” 2023. [Online]. Available: https://www.tensorflow.org/lite/microcontrollers

[24] J. Liu, Z. Zhao, and Y. Chen, “Wearable Vision for Blind Navigation,” *Sensors*, vol. 20, no. 21, 2020.